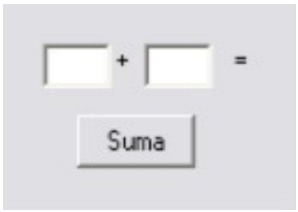
COLEXIO VIVAS S.L.	RAMA:	Informática	CICLO:	DAM		
	MÓDULO	Desarrollo de Interfaces/Programación de Servicios			CURSO:	2º
	PROTOCOLO .	Apuntes clases	AVAL:		DATA:	
	Autor	Francisco M. Bellas Aláez (aka Curro)				

Ejercicios temas 1, 2 y 3.

1ª Parte: Elementos básicos

Ejercicio 1

Crea un nuevo proyecto Windows Forms y en el formulario coloca 2 cajas de texto y 2 etiquetas de forma que quede de la siguiente manera:



Funcionará de la siguiente forma: El usuario introducirá números en cada casilla, y al darle al botón, aparecerá la suma en la misma etiqueta donde está el =. Recuerda que la propiedad Text de las TextBox es de tipo cadena.

Opcional: Realiza control de excepciones sabiendo que es prácticamente igual a como lo hacías en Java.

Ejercicio 2

Realizar un programa en la consola (Console App) que pida al usuario el nombre (string), la edad (int) y el peso (double). Debe mostrar en un único WriteLine y sin usa concatenación la siguiente información:

- En la primera linea: el nombre ocupando 12 caracteres, la edad ocupando 4.
- En la segunda línea un tabulador (carácter de escap correspondiente) y a continuación el peso ocupando 5 caracteres y con sólo 1 decimal.
- En la tercera línea Escribirá de nuevo el nombre entre comillas y a continuación la edad entre barras (\).

Ejercicios

Ejemplo:

```
Nombre:      Anakin, Edad:  30
     Peso: 75.5
"Anakin" \30\
```

Ejercicio 3

Realiza un programa en Windows Forms que simule una tragaperras. Mediante 3 TextBox establecer un Tragaperras con números. Existirá un botón y una etiqueta de información que indica si hay o no premio. Otra etiqueta indica el crédito disponible. Se parte de 50 euros. Una partida cuesta 2 euros. En cuanto se pulse el botón de juego en cada Textbox aparecerá un número aleatorio del 1 al 7. Si salen 3 números iguales se le indicará al usuario que hay premio de 20 euros, si salen 2 iguales hay premio de 5 euros. Un botón de crédito permite meter 10 euros más de crédito.

Debe existir al menos una función que devuelva (mediante tupla) 3 números aleatorios entre un mínimo y un máximo (incluidos) que se pasarán como parámetro. Úsala para obtener los aleatorios que luego se mostrarán.

Para facilitar la depuración se establecerá mediante directivas de compilación (Ver **apéndice III**) que si salen 2 iguales se resten 5 euros en lugar de ganarlos. El nombre de la directiva (en el define) será PRUEBA. Si PRUEBA existe resta y si no existe suma.

Ejercicio 4

Realizar en modo consola un menú con 3 opciones y Salir utilizando estructuración mediante funciones (al menos cada opción una función) y evitando repetir código.

Opción 1. Año bisiesto. Las acciones serán las siguientes:

- a) Pide un año al usuario: Debe ser positivo y menor que 10000.

- b) Llama a la función bisiesto. Esta tiene un parámetro entero que es el año y devuelve true si dicho año es bisiesto y false si no lo es). Busca el algoritmo (ojo, si se lo preguntas auna IA dile explícitamente que no te de una solución en seudcódigo u otro lenguaje de programación, si no, te lo hace todo y no es el objetivo).

- c) Muestra en pantalla si es o no bisiesto.

Opción 2. Suma de rango de números. Las acciones serán las siguientes:

- a) Se le piden al usuario 2 números positivos menores que 10000.

- b) Se llama a una función sumaRango que tiene dos parámetros. Dicha función suma los numeros entre el primer y el segundo parámetro. Devuelve null si el primer parámetro es mayor que el segundo o la suma de todos los números entre ambos parámetros en caso contrario.

- c) Muestra el resultado de la función. Si la función devuelve null indica el error por pantalla.

Opción 3. Todas las opciones: El programa pasará por los 2 apartados anteriores. No se puede llamar simplemente a funciones, es necesario jugar con **if, break y goto case.**

Opción 4. Salir. Mientras no se seleccione esta opción, el ordenador se mantendrá en el programa.

Opcional: El menú hazlo de forma que resalte la opción con un indicado o con video inverso y que se pueda seleccionar una u otra opción con los cursores (Usa ReadKey, SetCursorPosition, ForegroundColor,...).

Ejercicios

Ejercicio 5

Realiza un programa con las siguientes funciones y programa principal:

a) Función que se le pase un número y calcula el factorial del mismo. Si el número es negativo o mayor de 10 la función devuelve false. En caso contrario devuelve true. El resultado del factorial irá en un **parámetro por referencia.**

b) Función que dibuja en posiciones aleatorias de la pantalla la cantidad de asteriscos que se le pasa como parámetro. Lo harán entre las filas 1 y 10 y las columnas 1 y 20. Si no se le pasa ningún parámetro dibujará 10 asteriscos (Usa **parámetros opcionales,** no sobrecarga). Usa Console.SetCursorPosition para la colocación.

c) En el Main debe hacer código suficiente para probarlas cumpliendo que debe ejecutar una u otra dependiendo de una directiva de compilación.

Ejercicio 6 (Ver Apéndice I)

Realizar un pequeño programa en Windows Forms que mediante un TextBox pida un texto al usuario. Tras eso, el usuario deberá pulsar un botón y se le preguntará mediante una MessageBox con Icono de Pregunta si el usuario desea poner ese texto como título del formulario principal y si contesta que sí, cambiar dicho título (Text=titulo). El texto introducido debe repetirse en la MessageBox como parte del mensaje (concatenación).

2ª Parte: POO

Ejercicio 7

Se desea realizar un gestor de cuerpos celestes. Para ello se pide generar el siguiente esquema de clases:

a) **Clase Astro:**

- Es abstracta.
- Con propiedades públicas Nombre (string) y Radio (double).
- Al guardar el nombre lo hará en mayúsculas y lo devuelve entre comillas.
- Al guardar el radio se comprueba que sea positivo, si no fuera así lanza la excepción ArgumentException (Se hace exactamente igual que en Java).
- Tendrá un constructor que inicializa las propiedades a parámetros y otro sin parámetros que inicializa a "Tierra" y 6378 km. Sobreescribe Equals de forma que se considera que dos astros son iguales si son iguales sus nombres. Además si el parámetro es string y coincide con el nombre también devolverá true.

b) **Interface ITerraformable.** Dispone de la función esHabitable que no tiene parámetros y devuelve una booleana.

c) **Clase Planeta:**

- Hereda de Astro y dispone de una propiedad booleana denominada Gaseoso con set y get autoimplementados.
- Otra propiedad pública entera que indicará el n.º de satélites que tiene. Si se trata de guardar un valor negativo guardará 0.
- Tendrá dos constructores, uno inicializará todas las propiedades a

parámetros. Inicializa lo que se pueda llamando al constructor de Astro.

- El otro constructor sin parámetros que inicializa a "" nombre, 1 radio y false la propiedad gaseoso llamando al primer constructor. Cantidad de satélites a 0.

- Implementa la interface ITerraformable. Si no es gaseoso y el radio está entre 2000 y 8000km devuelve true. En caso contrario devuelve false.

- Sobreescribe ToString de forma que devuelva la propiedad Nombre ocupando 10 caracteres, la cantidad de satélites ocupando 4 caracteres y el radio con 2 decimales (usa string.Format o cadenas interpoladas si ya las has visto).

- Sobrecarga los operadores ++ y -- de forma que si se aplica a un Planeta, incremente o decremente en una unidad la cantidad de satélites. No debe permitir bajar de 0.

  Es decir, si se dispone del siguiente código:

```
Planeta p = new Planeta();
p++;
```

  Debe funcionar e incrementar el n.º de satélites de p. Análogamente con p-- debería de disminuir salvo que ya sea 0.

d) **Clase Cometa**:

- Hereda de Astro e implementa la interface ITerraformable de forma que siempre devuelve false.

Realiza un Main en una clase aparte para probar los distintos elementos programados.

Ejercicios

3ª Parte: Otros elementos de Java

A partir de este punto, es **obligatorio el control de excepciones.** No puede fallar nada.

Ejercicio 8 (Continuación del ejercicio 7, pero se valida por separado)

Este ejercicio es continuación del 7, de hecho se pide la realización de un Main que gestione objetos de las clases creadas previamente, por tanto hazlo en el mismo proyecto.

Haz un **programa principal** (Mételo en otra clase distinta a las clases anteriores) se crea una **colección de Astros** y el siguiente menú (Controla las excepciones de las opciones de menú con *TryParse*):

- Añade Planeta: Pregunta si es gaseoso y pide el nombre y radio. También pregunta cantidad de lunas.
- Añade Cometa: Pide su nombre y radio.
- Mostrar datos: para cada elemento indica si es terraformable. Además en el caso de Cometa mostrará el nombre y en el caso de Planeta mostrará lo que devuelva ToString.
- Incrementa/Decrementa n.º de satélites. Pide nombre del Astro y si existe (usa IndexOf) y es de tipo Planeta se pregunta si se desea incrementar o decrementar en una unidad el n.º de satélites.
- Eliminar no terraformables: Elimina todos los elementos que no sean terraformables de la colección.
- Salir. Como siempre no sale de la aplicación hasta que se selecciona esta opción.

Ejercicio 9

Define una clase Ordenador que contenga como propiedades Nombre y memoria RAM. Al asignarse RAM debe comprobar que sea mayor o igual a 0, en caso contrario lanzará excepción NegativeRAMException definida por ti. Añade constructor de inicialización.

En la misma clase crea una función estática que compruebe si una IP es válida. Se le pasa como parámetro un string y debe comprobar que consta de cuatro bytes separados por puntos. Si es correcto devuelve true, false en caso contrario.

Ya en otra clase define el Main con las siguientes características:

Crear una tabla hash (tipo Dictionary) para gestionar ordenadores de una red. Se usará como clave las IPs de ordenadores (string) y como valor objetos de tipo Ordenador.

Plantea un menú para:

1. Añadir nuevo equipo: Al pedir datos se debe comprobar que la IP es válida, que el nombre no esté vacío y que la cantidad de RAM es un entero positivo.

2. Añadir varios equipos: En una única petición se pueden meter varios equipos con este formato:

```
ip1:ram1,ip2:ram2,ip3:ram3,…
Por ejemplo:
192.0.0.1:16,192.0.0.2:8
```

En este caso usarás como nombre del ordenador el prefijo "NONAME" y le concatenarás la IP. P.ej. "NONAME_192.0.0.1".

3. Elimina un dato por clave.

4. Muestra la lista de nombres.

5. Muestra de un elemento de la colección (A partir de la clave muestra su nombre y su RAM o indica que no existe).

6. Salir

Por supuesto debes aplicar la modularidad adecuada.

Ejercicios

Ejercicio 10. Cada parte se valida por separado

1ª Parte: Clase Gestor Matriz

Se desea realizar una clase para gestionar matrices bidimensionales denominada **GestorMatriz**. Dispondrá de una propiedad tipo array bidimensional de enteros denominada Matriz y los siguientes métodos:

a) Un **constructor** con dos parámetros enteros que representan el número de filas y número de columnas. La propiedad Matriz se crea del n.º de filas y columnas indicado por los parámetros y se rellena con números aleatorios entre -5 y 20 ambos incluidos.

b) Sobrecargado otro constructor sin parámetros que, llamando al anterior, inicializa una tabla de 3 filas y 4 columnas.

c) Método **SumaFila**: se le pasa un parámetros entero y devuelve una tupla con un entero y un booleano. El booleano devuelve true si el parámetro es un índice válido y false si no lo es. En caso de índice válido el entero devuelto es la suma de los elementos de la fila. Si no devuelve 0.

d) Método **SumaColumna**. Se le pasa una parámetro entero que representa una columna y tendrá otro parámetro por referencia pero de salida (out) donde devuelve el resultado de la suma de los elementos de la columna. Devolverá true si el índice es correcto o false si no lo es.

e) Método **SumaMatriz**: tiene como parámetro un object. Si el object es un array bidimensional de int o double o un objeto del tipo GestorMatriz y es del mismo tamaño que Matriz, se suma elemento a elemento y el resultado se devuelve (como array bidimensional de double). En otro caso lanza una excepción creada por ti denominada MatrizException.

f) Método **Medias**: Tiene un parámetro booleano. Si es true devuelve un vector de doubles con la media de cada una de las filas. Si es false devuelve un vector de doubles con la media de cada una de las columnas.

g) Método **estático MostrarMatriz**: Parámetro tipo array bidimensional de cualquier tipo (usa genéricos<T>). Muestra la matriz parámetro teniendo como cabeceras de filas el número de fila (empezando en 1) y como cabecera de columna una letra Unicode empezando en la A. Debe quedar todo bien alineado. Usa 5 para ancho de columna.

2ª Parte: Clase Principal (Se valida por separado)

Dentro de otra clase, en el programa principal, crea un objeto tipo GestorMatriz denominado **gm** llamando al constructor vacío y haz un menú con las siguientes opciones usando las funciones realizadas:

1. Mostrar tabla.

2. Mostrar medias: Muestras las medias de filas y de columnas usando dos decimales para mostrar las medias. Debe quedar bien claro que es cada cosa que se muestra en pantalla.

3. Mostrar suma de una fila que se le pide al usuario. Indica si hay error.

4. Mostrar suma de una columna que se le pide al usuario. Indica si hay error.

5. Incremento aleatorio: Crea una array que tenga un n.º aleatorio **real** entre 1 y 10 en todas sus posiciones y del mismo tamaño que la matriz de gm. Llama a SumaMatriz pasando dicho array. Muestra ambas matrices y la matriz resultado para ver que todo es correcto.

6. Cambiar Matriz: Pide n.º de filas y n.º de columnas y crea un nuevo objeto que asigna a gm y trabajar con él.

7. Salir. No debe salir hasta que se seleccione esta opción.

OPCIONALES

OP1

Realiza el siguiente juego en Consola: Un meteorito se abalanza sobre la tierra y hay que destruirlo. El meteorito está representado por un asterisco que se mueve de izquierda a derecha empezando en la línea superior. Cada vez que llega al final de la línea, empieza en la línea siguiente, de forma que va bajando.

En la parte inferior tenemos un cañón que puede ser representado por una 'A' y se puede mover de izquierda a derecha y viceversa. Este cañón dispara solo hacia arriba. Solo puede efectuar un segundo disparo cuando el primero se haya salido de la pantalla.

Si el meteorito alcanza el tercio inferior de la pantalla, el jugador pierde. Se puede marcar dicho tercio inferior por una línea.

Se deben establecer mejoras: niveles (el meteorito va más rápido, dos meteoritos, …), número de vidas del jugador, puntuación según lo pronto que se acierte, que el meteorito y/o nave no sean un único carácter, etc…

NOTAS:

- Se recomienda el uso de la función Thread.Sleep para el control de tiempo.
- Para detectar la pulsación de una tecla, usar Console.KeyAvailable.
- Para leer el contenido de la tecla pulsada usa Console.ReadKey. Busca la forma para que no haga eco por pantalla.

Estructura genérica del programa

```
hacer
    hacer
        Movimiento de los objetos del juego (meteorito, disparo
        si se ha hecho, etc…) según control de tiempo.
        Detección de colisiones.
    Mientras no se pulse una tecla (KeyAvailable)
    Leer tecla pulsada (ReadKey)
    Gestión de la acción según la tecla pulsada (mover, disparar,
    salir,…)
mientras no acabe el juego
```

Ejercicios

OP2

Se trata de hacer una calculadora similar a la de windows (modo gráfico) pero más sencilla. Se recomienda ir por pasos pues no es tan trivial como puede parecer:

Primero establecer botones de cifras, pantalla, una o dos operaciones y botón de cero. En principio no se trabajará con decimales por parte del usuario (sí para resultados).

Cuando lo anterior funcione bien puedes añadir el resto de las operaciones.

Ya luego la amplías a tu gusto: Añadir memorias parciales, añadir tecla delete, decimales, etc...

OP3

a) Realiza una función a la que se le pase de forma obligatoria un símbolo de suma o multiplicación y luego una ristra de tamaño indeterminado de números enteros. Debe realizar la cuenta con todos los números y devolver el resultado (como función, no por pantalla). Si el símbolo de operación que se le pasa es incorrecto, no hará nada.

Realiza un programa que pruebe la función anterior (no es necesario realizar interfaz con el usuario, se pueden probar ejecutándola directamente.)

b) Realizar un programa similar a la función del apartado (a) pero que pueda ser ejecutado desde la línea de comando y muestre el resultado. Es decir, se puede ejecutar así:

```
C:\>  calcula + 5 6 4 10
```

y muestre el 25. Por supuesto la lista de números será indeterminada.

OP4

Basándote en la idea el ejercicio del meteorito, realizaun juego con elementos móviles en consola pero ya más complejo: niveles, distintas pantalla, puede haber escenario. Te recomiendo un tipo space invaders, vista de pájaro, plataformas...

OP5

Mastermind: El ordenador saca una combinación de cuatro símbolos (o colores) de siete posibles y el usuario tiene que adivinarlo. En cada turno el usuario da una combinación y el ordenador le dice cuantos aciertos de símbolo hay y cuantos aciertos de símbolo y posición hay. Usa vectores de 4 elementos. Se deben guardar

todas las tiradas del usuario en una colección para hacer un informe final de número de tiradas, momento de mayor acierto (salvo el final si gana el usuario), momento de meno aciertoy otras que se te ocurran. Se debe poder abandonar el juego en la mitad que significaría que pierde el usuario. Puedes intentar hacerlo en modo gráfico.

Más información en: http://es.wikipedia.org/wiki/Mastermind

OP6

Cualquier otro juego de tablero que se te ocurra.